

Experiment-07

Develop a Django app that performs student registration to a course. It should also display list of students registered for any selected course. Create students and course as models with enrolment as ManyToMany field.

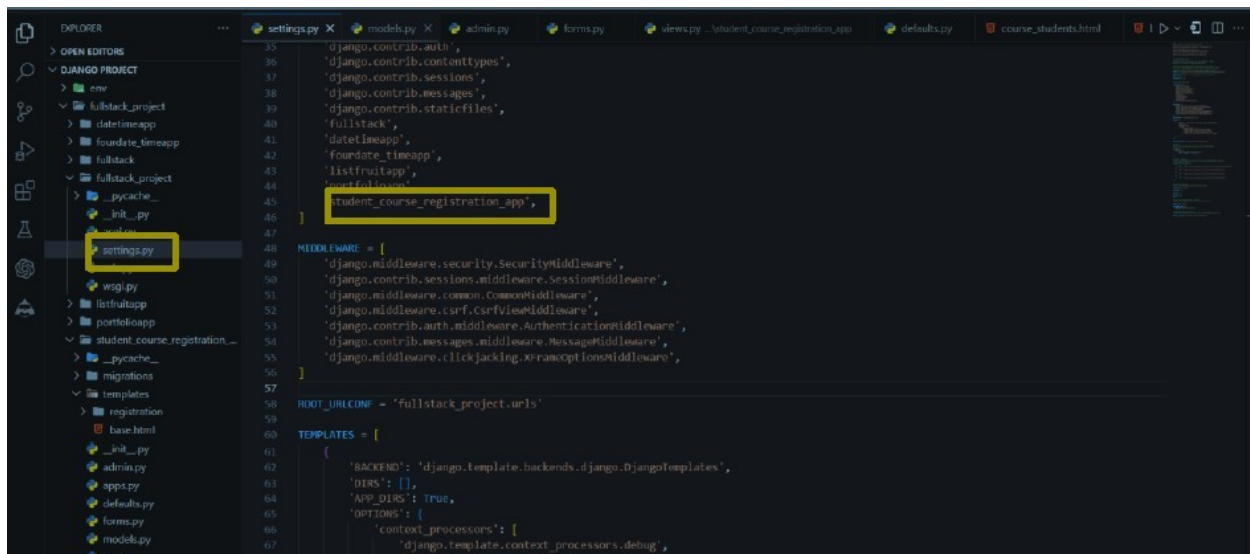
Step-01: Create project.

Step-02: Change directory and Create App. (Eg. App name: student_course_registration_app)

Step-03: Add the app to INSTALLED_APPS

Open the settings.py file in your project's directory.

Locate the **INSTALLED_APPS** list and add the name of your new app to the list:



Step-04: Create models

- Open the **models.py** file inside the **student_course_registration_app** and define your models:

from django.db import models

class Course(models.Model):

name = models.CharField(max_length=255)

```
description = models.TextField(blank=True, null=True)
```

```
def __str__(self):
```

```
    return self.name
```

```
class Student(models.Model):
```

```
    first_name = models.CharField(max_length=255, default='')
```

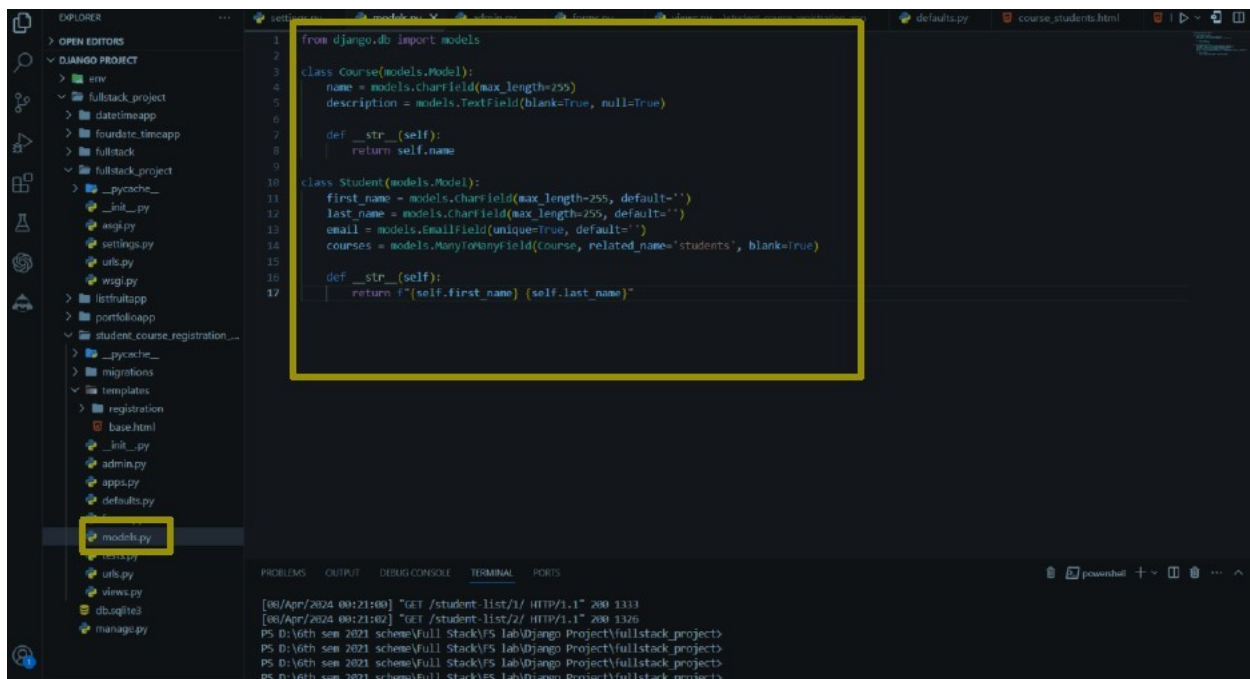
```
    last_name = models.CharField(max_length=255, default='')
```

```
    email = models.EmailField(unique=True, default='')
```

```
    courses = models.ManyToManyField(Course, related_name='students', blank=True)
```

```
def __str__(self):
```

```
    return f'{self.first_name} {self.last_name}'
```



Step-05: Register models in the admin site

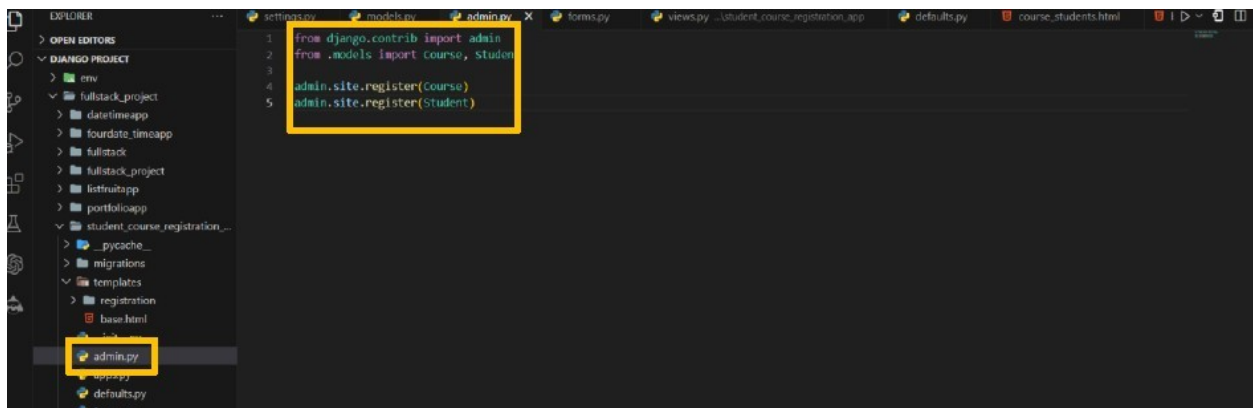
- Open the admin.py file inside the student_course_registration_app and register your models:

```
from django.contrib import admin
```

```
from .models import Course, Student
```

```
admin.site.register(Course)
```

```
admin.site.register(Student)
```

**Step-06: Create forms**

- Create a new file called **forms.py** inside the student_course_registration_app and define your forms:

```
from django import forms
```

```
from .models import Student, Course
```

```
class StudentForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Student
```

```
        fields = ['first_name', 'last_name', 'email', 'courses']
```

```
        widgets = {
```

```
            'first_name': forms.TextInput(attrs={'class': 'form-control'}),
```

```
'last_name': forms.TextInput(attrs={'class': 'form-control'}),  
  
'email': forms.EmailInput(attrs={'class': 'form-control'}),  
  
'courses': forms.SelectMultiple(attrs={'class': 'form-select', 'data-toggle': 'dropdown'}),  
  
}
```

```
class CourseForm(forms.ModelForm):
```

```
    class Meta:
```

```
        model = Course
```

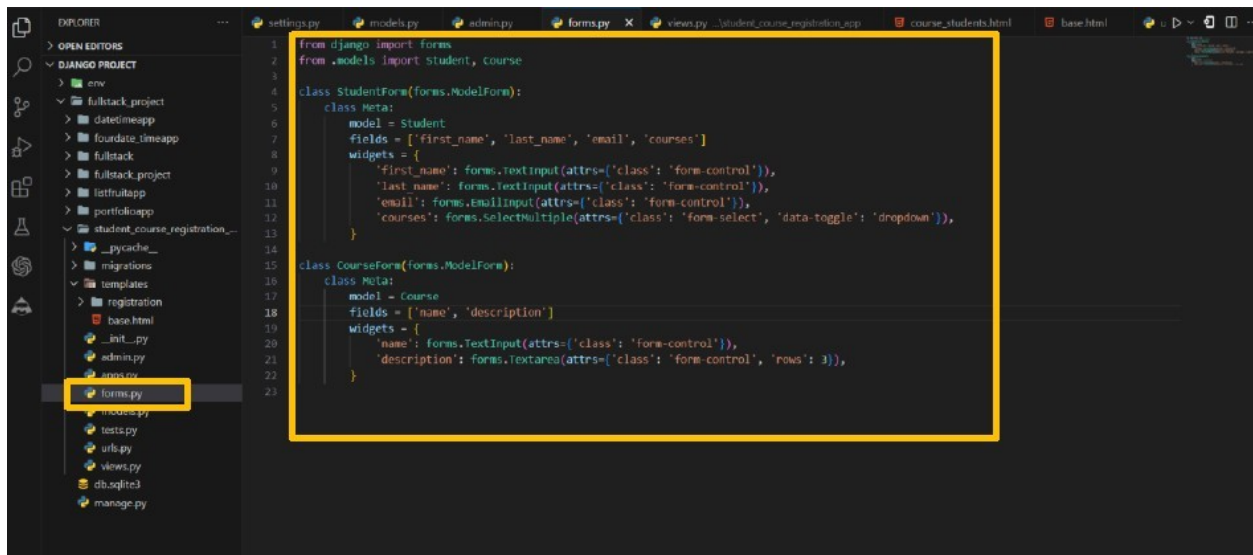
```
        fields = ['name', 'description']
```

```
        widgets = {
```

```
            'name': forms.TextInput(attrs={'class': 'form-control'}),
```

```
            'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 3}),
```

```
        }
```



Step-07: Create views

- Open the **views.py** file inside the `student_course_registration_app` and define your views:

```
from django.shortcuts import render, redirect

from .models import Course, Student

from .forms import StudentForm, CourseForm

def index(request):

    courses = Course.objects.all()

    return render(request, 'registration/index.html', {'courses': courses})

def register_student(request):

    if request.method == 'POST':

        form = StudentForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('index')

    else:

        form = StudentForm()

    return render(request, 'registration/register_student.html', {'form': form})

def register_course(request):

    if request.method == 'POST':

        form = CourseForm(request.POST)

        if form.is_valid():

            form.save()

            return redirect('index')

    else:
```

```

        form = CourseForm()

    return render(request, 'registration/register_course.html', {'form': form})

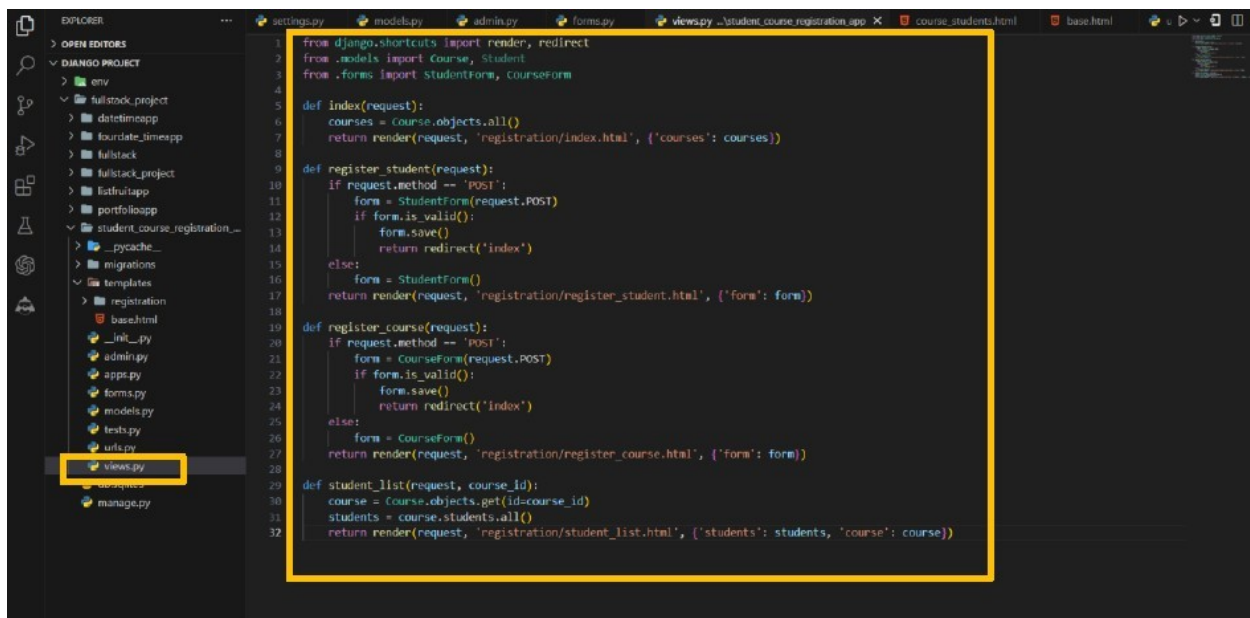
def student_list(request, course_id):

    course = Course.objects.get(id=course_id)

    students = course.students.all()

    return render(request, 'registration/student_list.html', {'students': students, 'course':
course})

```



Step-08: Create templates

- Create a new directory called templates inside your student_course_registration_app, and create the following template files:

index.html

```
{% extends 'base.html' %}
```

```
{% block content %}
```

```
<div class="container">
```

```
<h1 class="mt-5">Courses</h1>
```

```
<ul class="list-group mt-3">
```

```
    {% for course in courses %}
```

```
        <li class="list-group-item"><a href="{% url 'student_list' course.id %}">{{ course.name }}</a></li>
```

```
    {% endfor %}
```

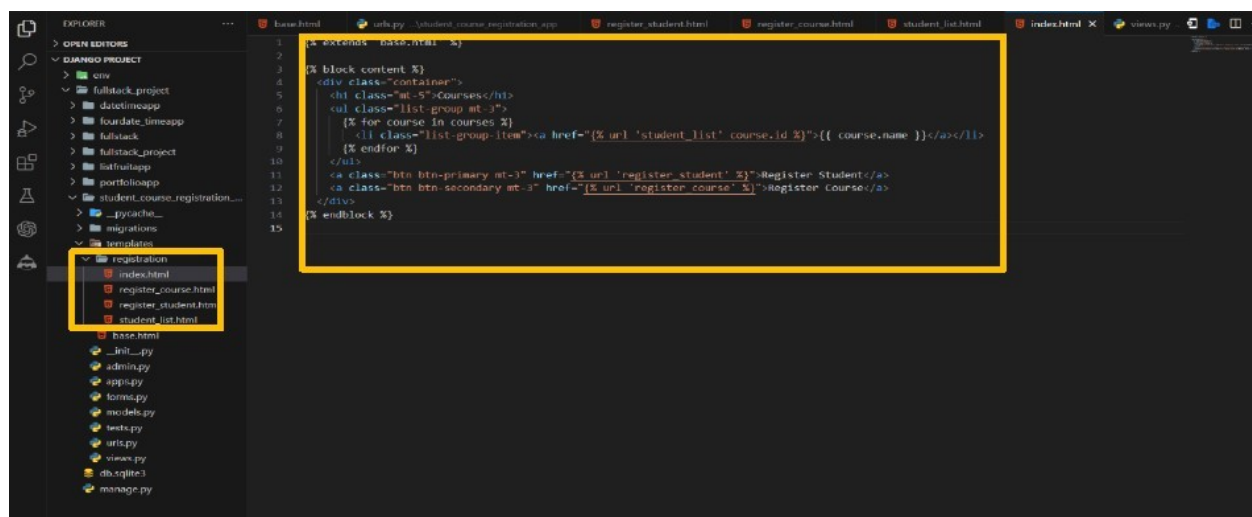
```
</ul>
```

```
<a class="btn btn-primary mt-3" href="{% url 'register_student' %}">Register Student</a>
```

```
<a class="btn btn-secondary mt-3" href="{% url 'register_course' %}">Register Course</a>
```

```
</div>
```

```
{% endblock %}
```



register_student.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Student Registration</title>

<!-- Bootstrap CSS -->

<link                                href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/css/bootstrap.min.css" rel="stylesheet">

</head>

<body>

<div class="container">

    <h1>Register Student</h1>

    <form method="post">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit" class="btn btn-primary">Register</button>

    </form>

</div>

<!-- Bootstrap JavaScript (Optional) -->

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-
alpha1/dist/js/bootstrap.bundle.min.js"></script>

<!-- Your custom JavaScript to initialize the dropdown (Optional) -->

<script>

    // Example JavaScript to initialize the dropdown

    var dropdownElementList = [].slice.call(document.querySelectorAll('.dropdown-toggle'))

    var dropdownList = dropdownElementList.map(function (dropdownToggleEl) {

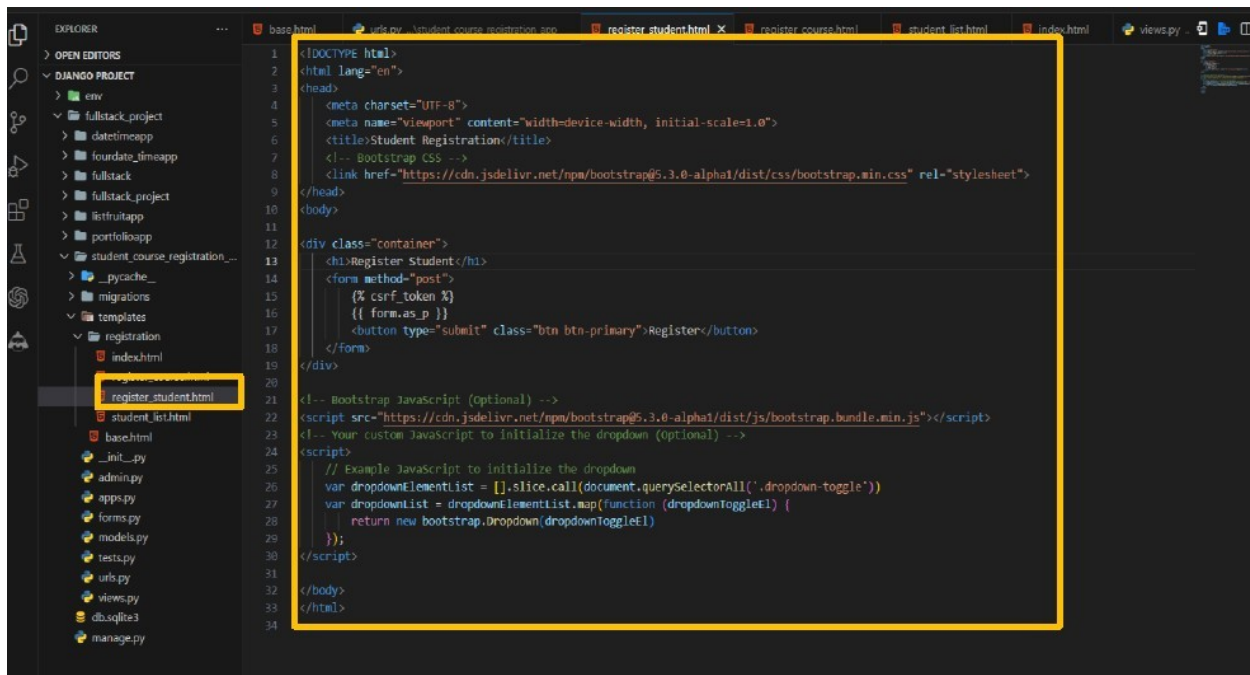
        return new bootstrap.Dropdown(dropdownToggleEl)

    });

</script>

</body>
```


</html>

**register_course.html**

{% extends 'base.html' %}

{% load static %}

{% block content %}

<h1>Register Course</h1>

<div class="container">

```

        {% if field.help_text %}

            <small class="form-text text-muted">{{
                field.help_text }}</small>

        {% endif %}

        {% for error in field.errors %}

            <div class="alert alert-danger">{{ error
                }}</div>

        {% endfor %}

    </div>

    {% endfor %}

    <button type="submit" class="btn btn-primary text-
center">Register</button>

</form>

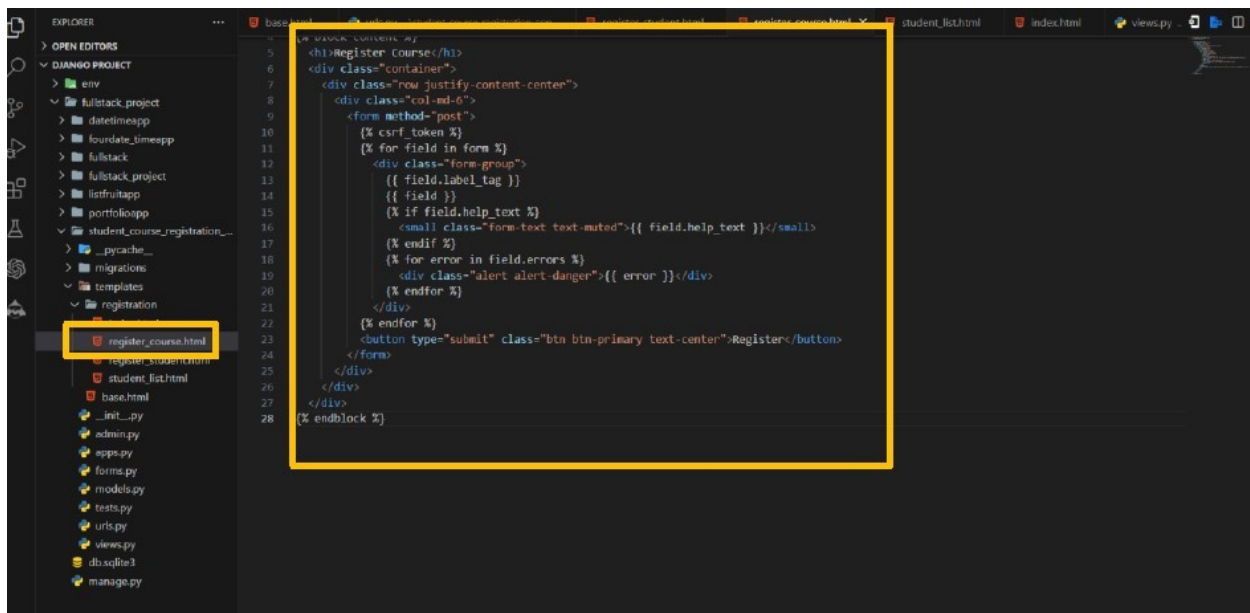
</div>

</div>

</div>

{% endblock %}

```



student_list.html

```
{% extends 'base.html' %}

{% block content %}

    <h1>Students Registered for {{ course.name }}</h1>

    <ul>

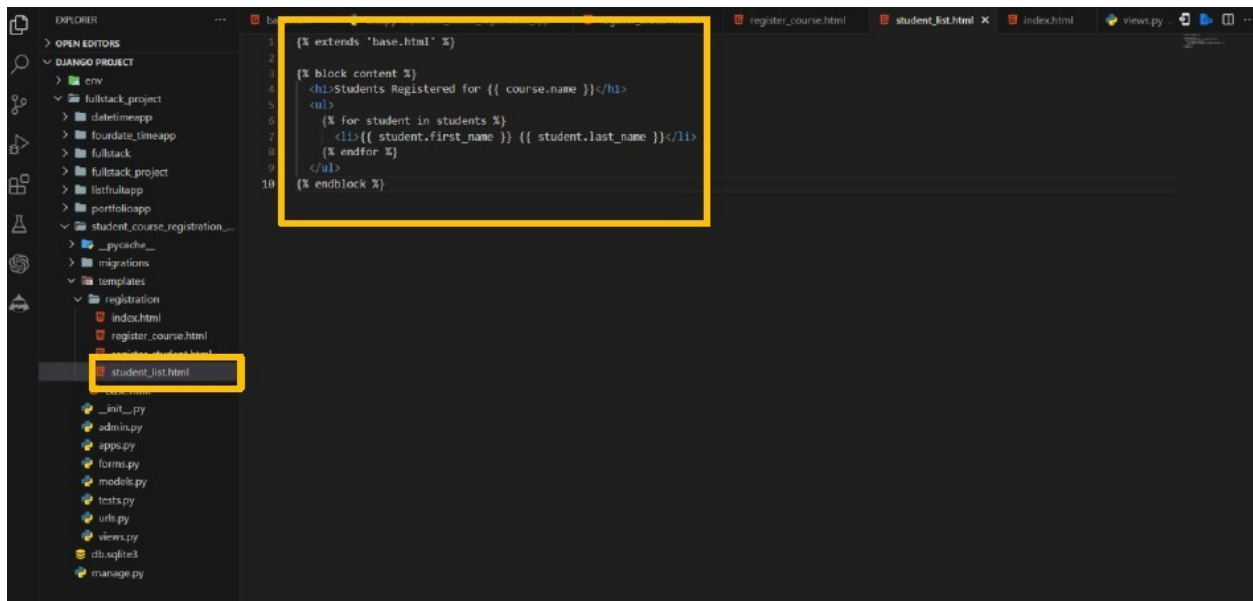
        {% for student in students %}

            <li>{{ student.first_name }} {{ student.last_name }}</li>

        {% endfor %}

    </ul>

{% endblock %}
```

**Step-09: Create a base template**

- Create a **base.html** file inside the templates directory with the following content:

base.html

```
<!DOCTYPE html>

<html>
```

```

<head>

<title>Student Registration</title>

</head>

<body>

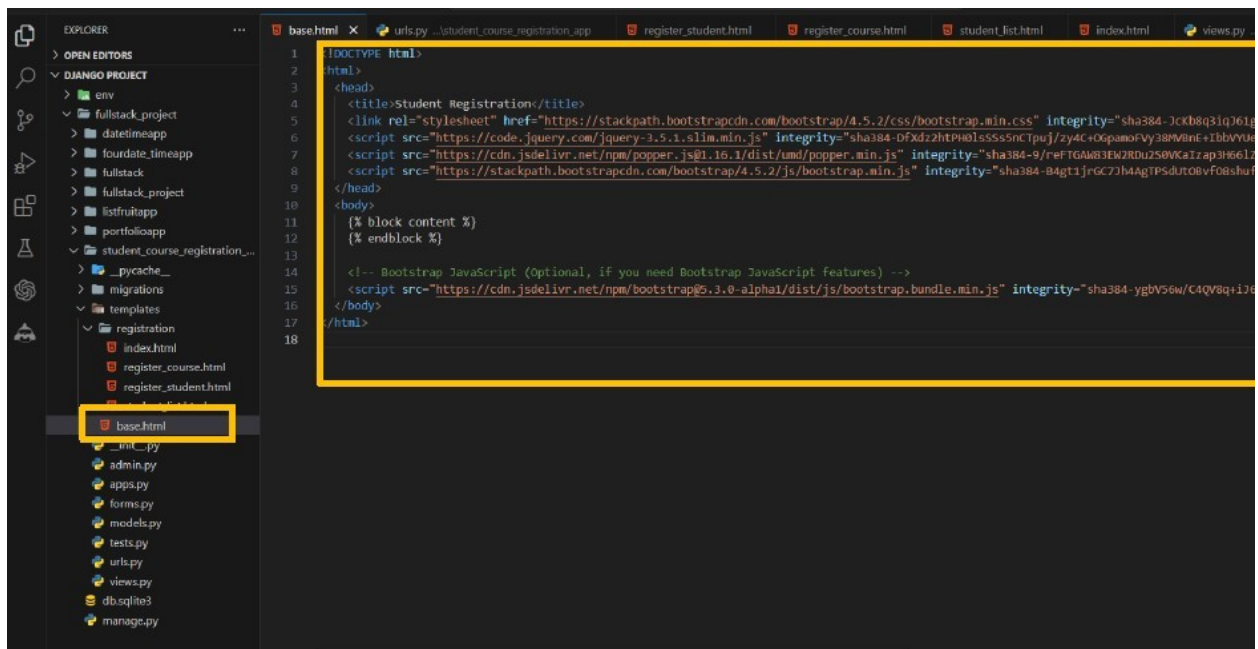
    {% block content %}

    {% endblock %}

</body>

</html>

```



Step-10: Configure URLs

- Open the urls.py file inside your student_course_registration_app and define your URLs:

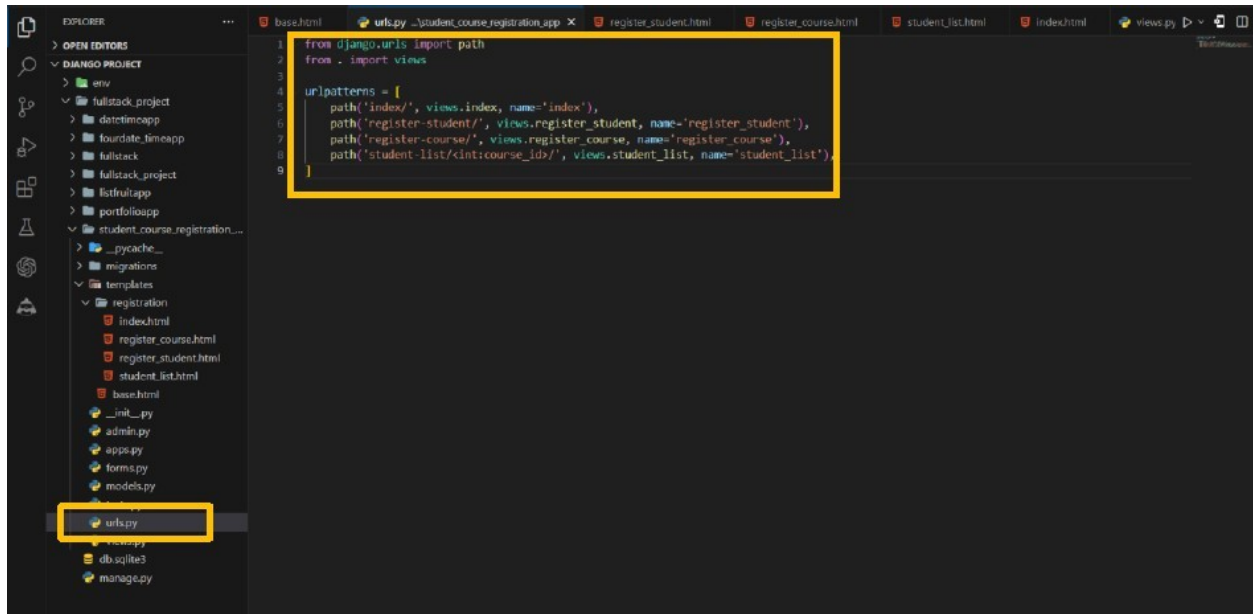
```
from django.urls import path
```

```
from . import views
```

```
urlpatterns = [
```

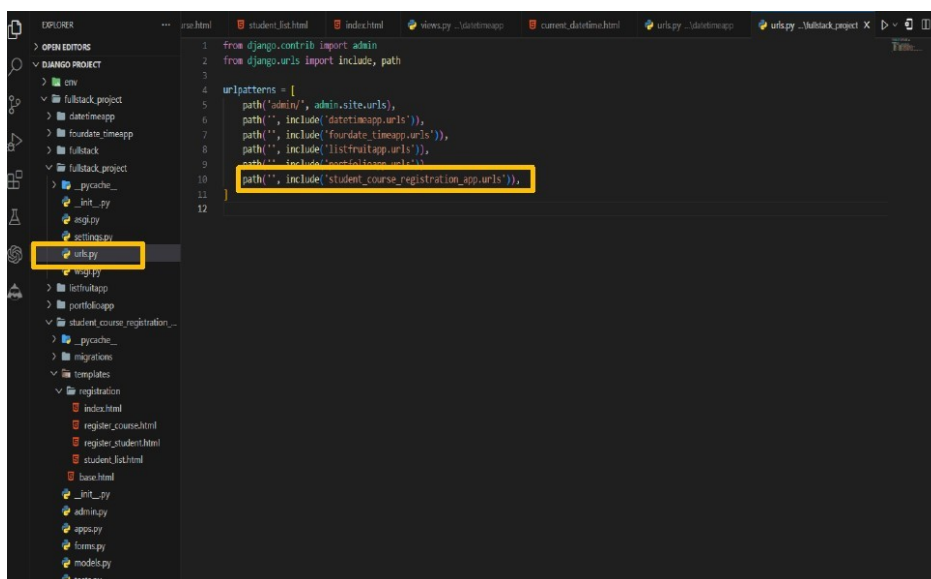
```
    path('index/', views.index, name='index'),
```

```
path('register-student/', views.register_student, name='register_student'),  
path('register-course/', views.register_course, name='register_course'),  
path('student-list/<int:course_id>', views.student_list, name='student_list'),  
]
```



Step-11: Update project URLs

- Open the urls.py file inside your project and include the URLs from the student_course_registration_app:



Step-12: Make Migration for check Models saved or not into the database

```
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
Migrations for 'student_course_registration_app':
  student_course_registration_app\migrations\0002_remove_student_name_student_courses_and_more.py
  - Remove field name from student
  - Add field courses to student
  - Add field first_name to student
  - Add field last_name to student
  - Alter field description on course
  - Alter field name on course
  - Alter field email on student
  - Delete model Enrollment
```

Step-13: Migrate To Save the Models into database

```
PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, student_course_registration_app
Running migrations:
  Applying student_course_registration_app.0002_remove_student_name_student_courses_and_more... OK
```

Step-14: Run the development server

- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.
- Type or copy this **http://127.0.0.1:8000/index/**

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

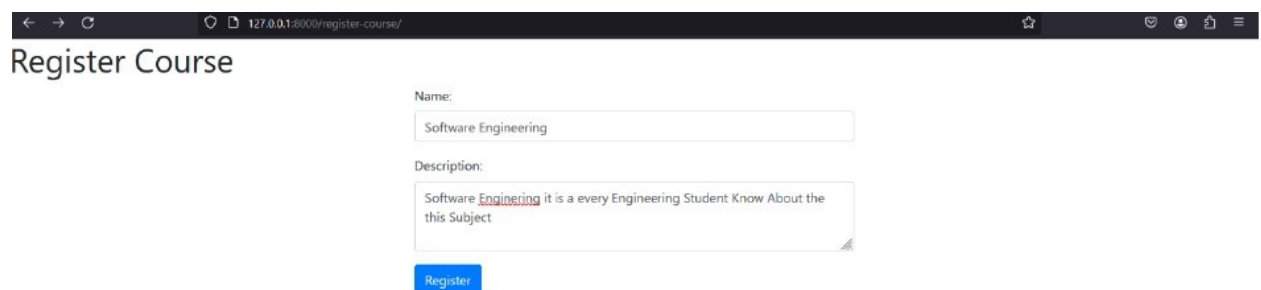
[]
```

Final Output By Clicking Register Student and Register Course

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/index/". The page title is "Courses". Below the title, there are two input fields: the first contains "DBMS" and the second contains "CN". At the bottom of the form, there are two buttons: "Register Student" (highlighted in blue) and "Register Course" (highlighted in grey).

Fig: Index Screen

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/register-student/". The page title is "Register Student". The form contains the following fields: "First name:" with the value "Hanumanthu", "Last name:" with the value "Hanu", "Email:" with the value "hanu@gmail.com", and "Courses:" with a list containing "DBMS" (highlighted) and "CN". A blue "Register" button is located at the bottom left of the form.

Fig: Student Register Screen

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:8000/register-course/". The page title is "Register Course". The form contains the following fields: "Name:" with the value "Software Engineering", and "Description:" with the value "Software Engineering it is a every Engineering Student Know About the this Subject". A blue "Register" button is located at the bottom left of the form.

Fig: Course Register Screen



Fig: Students Register Particular Courses Screen [You can Check as You Register Courses]

Experiment-08

For student and course models created in Lab experiment for Module2, register admin interfaces, perform migrations and illustrate data entry through admin forms.

Step-01: Register models in the admin site

```
from django.contrib import admin

from .models import Course, Student

@admin.register(Course)

class CourseAdmin(admin.ModelAdmin):

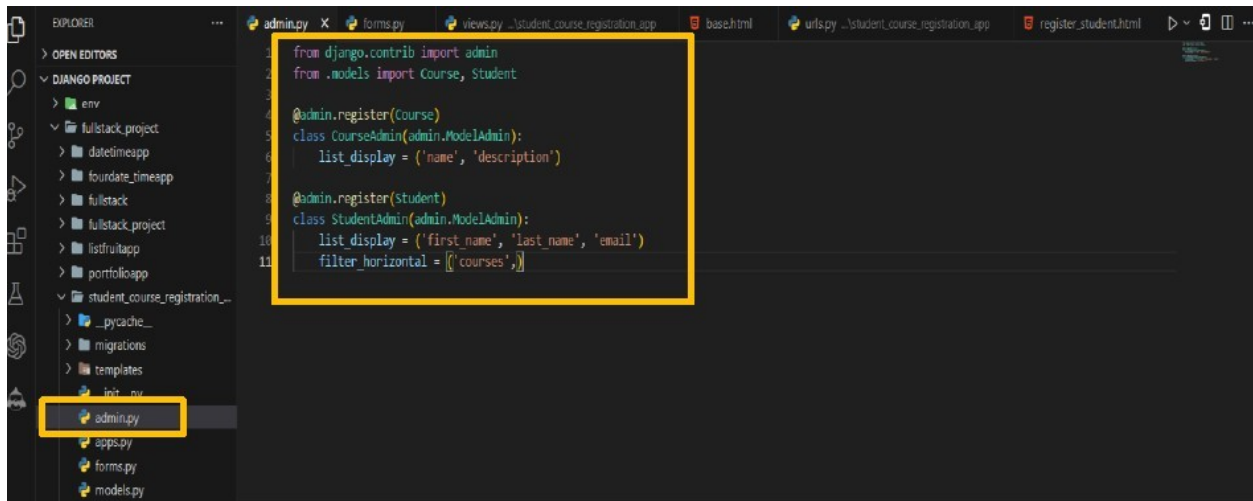
    list_display = ('name', 'description')

@admin.register(Student)

class StudentAdmin(admin.ModelAdmin):

    list_display = ('first_name', 'last_name', 'email')

    filter_horizontal = ('courses',)
```



- In this code, we've registered the Course and Student models with the Django admin site.
- We've also specified the fields to be displayed in the list view for each model using the list_display attribute.
- For the StudentAdmin class, we've added the filter_horizontal attribute to display the courses field (which is a many-to-many relationship) as a horizontal filter in the admin interface.

Step-02: Create and apply migrations

python manage.py makemigrations

```
no such file or directory
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project> cd fullstack_project
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py makemigrations
No changes detected
```

python manage.py migrate

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, student_course_registration_app
Running migrations:
  No migrations to apply.
```

Step-03: Create a superuser

- If you haven't already created a superuser for your project, you'll need to do so to access the admin interface.
- In your terminal or command prompt, run the following command

python manage.py createsuperuser

```
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py createsuperuser
Username (leave blank to use 'aryah'): admin
Email address:
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Type Username: admin

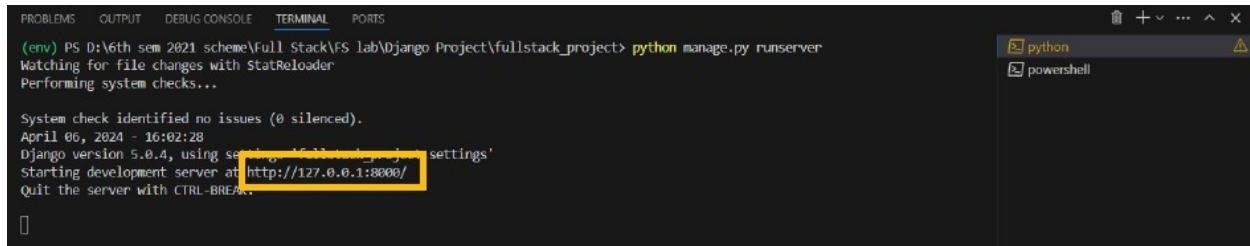
Password:1234 [For Simply but your wish to create your own username and password]

Step-04: Access the admin interface

- Start the Django development server by running the following command:
- In the VS Code terminal, navigate to your Django project's directory (the one containing manage.py).
- Run the development server

python manage.py runserver

- Open your web browser and visit **http://127.0.0.1:8000/**.



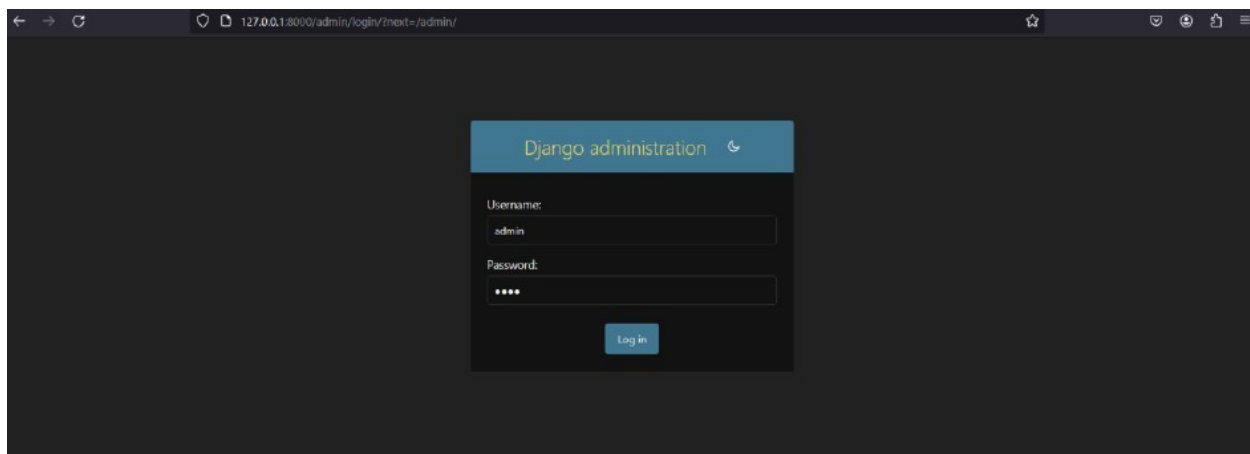
```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(env) PS D:\6th sem 2021 scheme\Full Stack\FS lab\Django Project\fullstack_project> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
April 06, 2024 - 16:02:28
Django version 5.0.4, using settings 'fullstack_project.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

- Type or copy this **<http://127.0.0.1:8000/admin/>**

Step-05: Log in to the admin interface

- Back in your web browser, enter the superuser credentials you just created and log in to the admin interface.



Step-06: Add data through admin forms

Once you're logged in to the admin interface, you'll see the "Courses" and "Students" sections in the left sidebar. Click on "Courses" to add a new course.

- Click on the "Add course" button in the top-right corner.
- Fill in the "Name" and "Description" fields for the new course.
- Click the "Save" button to create the new course.

Next, click on "Students" in the left sidebar to add a new student.

- Click on the "Add student" button in the top-right corner.
- Fill in the "First name", "Last name", and "Email" fields for the new student.
- In the "Courses" section, select the courses you want to enroll the student in by checking the appropriate boxes.
- Click the "Save" button to create the new student.

You can repeat these steps to add more courses and students through the admin interface. Additionally, you can view, edit, and delete existing courses and students from the admin interface.

Final Output

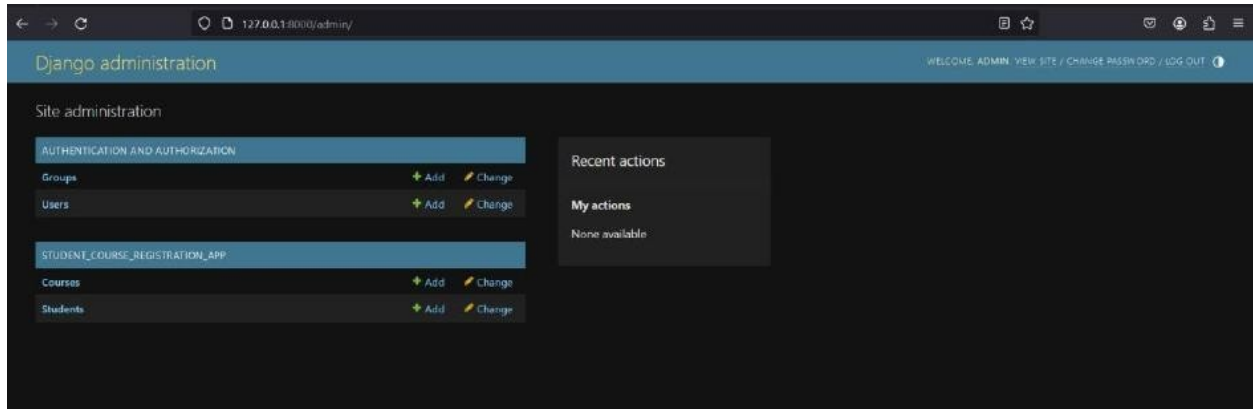


Fig: Admin Main Screen

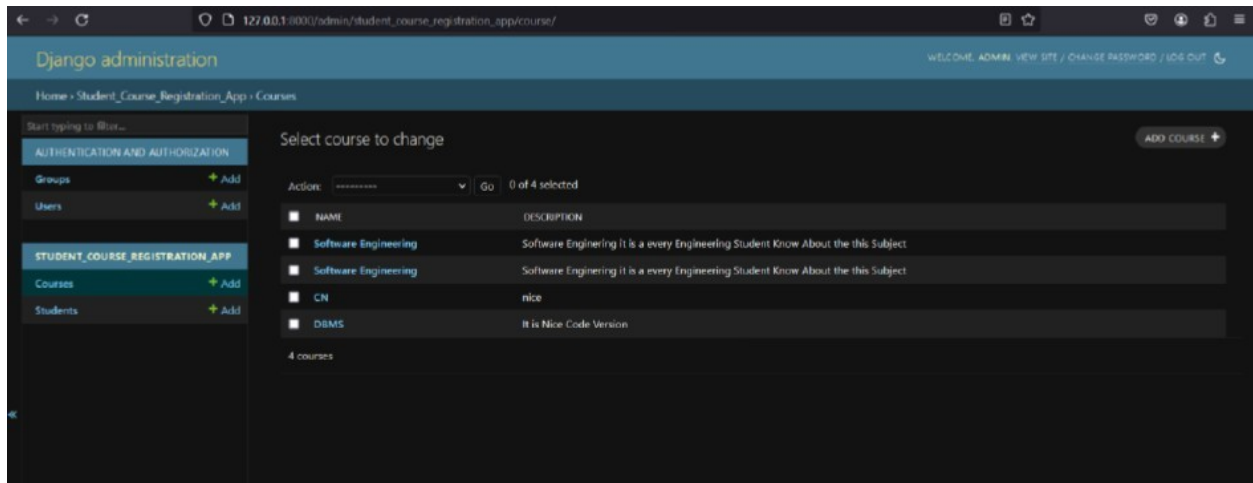
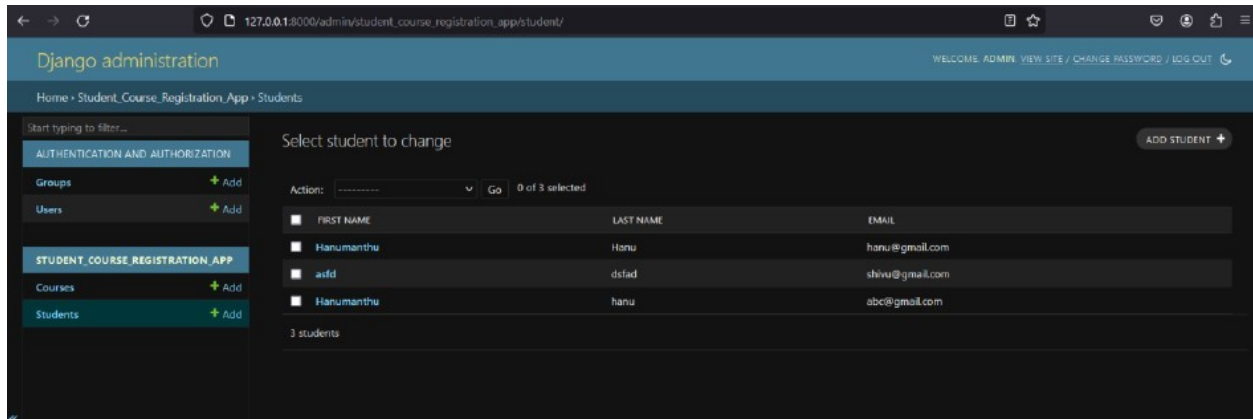


Fig: Admin Course Screen

**Fig: Admin Student Screen**